

# CRC Error Detection Simulator

## Data Communication & Networks Project

### Team Members:

1. Asmita Chowdhury (27631724002) (Leader)
2. Subhajit Mishra (27631724012)
3. Sukanya Biswas (27631724015)

**Faculty :** Prof. Sudipta Maity Sahu

**Semester:** 4<sup>th</sup> semester

**Department:** Computer Science & Engineering (Cyber Security)

**Academic Year:** 2025-2026



**BUDGE BUDGE INSTITUTE OF TECHNOLOGY**  
**Department of Computer Science & Engineering (Cyber**  
**Security)**

## Table Of Content

| <b>Sl No.</b> | <b>Content</b>                                   | <b>Page No.</b> |
|---------------|--------------------------------------------------|-----------------|
| <b>1.</b>     | Problem Statement & Introduction                 | 3               |
| <b>2.</b>     | Application Area                                 | 4               |
| <b>3.</b>     | Flow Diagram                                     | 5               |
| <b>4.</b>     | Code                                             | 6               |
| <b>5.</b>     | Output with Explanation                          | 7               |
| <b>6.</b>     | Working Principle                                | 8               |
| <b>7.</b>     | Role Division Among Group Members                | 9               |
| <b>8.</b>     | Learning Outcomes                                | 9               |
| <b>9.</b>     | Date and Time Evidences During Project Execution | 10              |
| <b>10.</b>    | Group Photos with Date                           | 10              |
| <b>11.</b>    | GitHub Link                                      | 10              |
| <b>12.</b>    | CO-PO Mapping                                    | 11              |
| <b>13.</b>    | Conclusion                                       | 11              |
| <b>14.</b>    | References                                       | 12              |

# 1. Problem Statement & Introduction

## Problem Statement

Data transmitted over communication channels is susceptible to errors caused by noise, interference, and hardware malfunctions. Undetected transmission errors can lead to incorrect information being received, resulting in communication failures and data loss.

The objective of this project is to develop a CRC Error Detection Simulator that implements the Cyclic Redundancy Check algorithm to detect errors in transmitted binary data. The system should accept input data and a generator polynomial, generate the CRC code, create the transmitted frame, and verify the received frame. The simulator must determine whether the received data is error-free or contains transmission errors by checking the CRC remainder.

## Introduction

In modern digital communication systems, data is transmitted between devices through various communication channels. During transmission, data may become corrupted due to noise, interference, signal distortion, or hardware failures. Therefore, reliable error detection techniques are essential to ensure data integrity and accuracy.

This project, "CRC Error Detection Simulator," demonstrates the working principle of CRC through a software implementation in Python. The simulator generates CRC bits, appends them to the data, simulates data transmission, and verifies the received data to detect transmission errors. The project provides practical understanding of error detection mechanisms used in networking, telecommunications, storage devices, and embedded systems.

## 2. Application Area (30 Minutes)

CRC is widely used in:

### 1. **Computer Networks**

- Ethernet
- Wi-Fi
- TCP/IP communication

### 2. **Storage Devices**

- Hard disks
- SSDs
- CDs/DVDs

### 3. **Embedded Systems**

- Microcontrollers
- IoT devices

### 4. **Telecommunication**

- Mobile networks
- Satellite communication

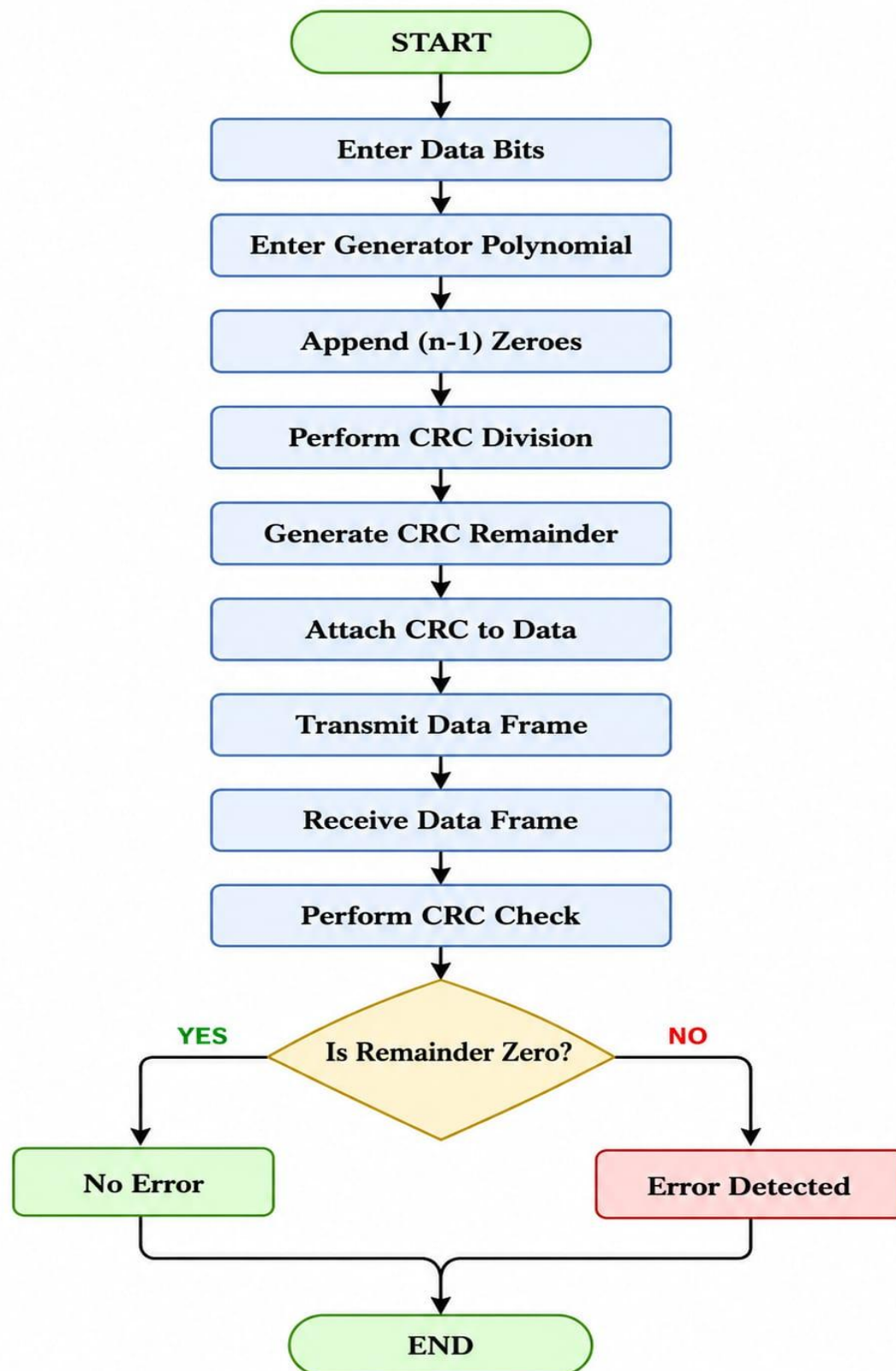
### 5. **Data Transmission Systems**

- Serial communication
- Wireless communication

## **Advantages**

- Fast error detection.
- High reliability.
- Low computational cost.
- Easy implementation.

### 3. Flow Diagram (1 Hour)



## 4. Code (3 Hours)

```

1 def ver(a, b):
2     result = ""
3     for i in range(1, len(b)):
4         if a[i] == b[i]:
5             result += "0"
6         else:
7             result += "1"
8     return result
9
10
11 def crc_division(dividend, divisor):
12     pick = len(divisor)
13     temp = dividend[0:pick]
14
15     while pick < len(dividend):
16         if temp[0] == '1':
17             temp = xor(divisor, temp) + dividend[pick]
18         else:
19             temp = xor('0' * pick, temp) + dividend[pick]
20         pick += 1
21
22     if temp[0] == '1':
23         temp = xor(divisor, temp)
24     else:
25         temp = xor('0' * pick, temp)
26
27     return temp
28
29
30 def generate_crc(data, divisor):
31     appended_data = data + '0' * (len(divisor) - 1)
32     remainder = crc_division(appended_data, divisor)
33     codeword = data + remainder
34     return remainder, codeword
35
36
37 def check_crc(received_data, divisor):
38     remainder = crc_division(received_data, divisor)
39
40     if set(remainder) == {'0'}:
41         return True
42     return False
43

```

```

43
44
45 # Main Program
46 print("==== CRC Error Detection Simulator =====")
47
48 data = input("Enter Data Bits: ")
49 divisor = input("Enter Generator Polynomial: ")
50
51 remainder, transmitted_data = generate_crc(data, divisor)
52
53 print("\nCRC Remainder :", remainder)
54 print("Transmitted Data :", transmitted_data)
55
56 received_data = input("\nEnter Received Data: ")
57
58 if check_crc(received_data, divisor):
59     print("\nNo Error Detected!")
60 else:
61     print("\nError Detected!")

```

## 5. Output with Explanation (1 Hour)

### Case 1: No Error During Transmission

#### Output

```

==== CRC Error Detection Simulator =====
Enter Data Bits: 1101011011
Enter Generator Polynomial: 10011

CRC Remainder : 1110
Transmitted Data : 11010110111110

Enter Received Data: 11010110111110

No Error Detected!

=== Code Execution Successful ===

```

#### Explanation

- Receiver performs CRC division.
- Remainder becomes all zeros.
- Therefore, transmitted data is correct.

## Case 2: Error During Transmission

### Output

```
===== CRC Error Detection Simulator =====  
Enter Data Bits: 1101011011  
Enter Generator Polynomial: 10011  
  
CRC Remainder : 1110  
Transmitted Data : 11010110111110  
  
Enter Received Data: 11010100111110  
  
Error Detected!  
  
=== Code Execution Successful ===
```

### Explanation

- One bit was modified.
- CRC division produces non-zero remainder.
- Error is successfully detected.

## 6. Working Principle

1. Append  $(n-1)$  zeros to the data, where  $n$  is the length of the generator polynomial.
2. Perform modulo-2 division (XOR division).
3. The remainder obtained is the CRC.
4. Append the CRC to the original data to create the transmitted frame.
5. At the receiver side, divide the received frame by the same generator polynomial.
6. If the remainder is all zeros, the data is accepted; otherwise, an error is detected.



**Time Complexity:**  $O(n \times m)$

- $n$  = length of data bits
- $m$  = length of generator polynomial

**Space Complexity:**  $O(n)$

## 7. Role Division Among Group Members (30 Minutes)

| Member   | Responsibility                            |
|----------|-------------------------------------------|
| Member 1 | Research, Project Design, Coding, Testing |
| Member 2 | Documentation and Data Collection         |
| Member 3 | Documentation Support and Presentation    |

## 8. Learning Outcomes (30 Minutes)

After completing this project, students will be able to:

1. Understand CRC error detection mechanism.
2. Learn binary division using XOR operations.
3. Implement communication protocols.
4. Develop programming skills in C.
5. Analyse algorithm complexity.
6. Work effectively in teams.
7. Prepare technical documentation.

## 9. Date and Time Evidences During Project Execution (30 Minutes)

| Activity             | Date       | Time     |
|----------------------|------------|----------|
| Topic Selection      | 10/06/2026 | 10:00 AM |
| Requirement Analysis | 10/06/2026 | 11:00 AM |
| Flowchart Design     | 11/06/2026 | 02:00 PM |
| Coding Phase         | 12/06/2026 | 10:00 AM |
| Testing Phase        | 13/06/2026 | 01:00 PM |
| Documentation        | 14/06/2026 | 03:00 PM |
| Final Submission     | 18/06/2026 | 11:00 AM |

## 10. Group Photos with Date (30 Minutes)



## 11. GitHub Link (1 Hour)

<https://github.com/asmitascript/crc-error-ditector>

## 12. CO-PO Mapping (30 Minutes)

| Course Outcome (CO)                        | Program Outcome (PO) | Mapping Level |
|--------------------------------------------|----------------------|---------------|
| CO1: Understand error detection techniques | PO1                  | High (3)      |
| CO2: Apply programming concepts            | PO2                  | High (3)      |
| CO3: Design communication solutions        | PO3                  | Medium (2)    |
| CO4: Analyze algorithm efficiency          | PO4                  | Medium (2)    |
| CO5: Team collaboration                    | PO9                  | High (3)      |
| CO6: Technical report writing              | PO10                 | High (3)      |

## 13. Conclusion

The **CRC Error Detection Simulator** successfully demonstrates the working principle of Cyclic Redundancy Check used in communication systems. The project verifies how CRC detects transmission errors efficiently using polynomial division and XOR operations. Through this project, students gain practical knowledge of data communication, networking concepts, and algorithm implementation.

## 14. References

### Books

1. Data Communications and Networking – Behrouz A. Forouzan.
2. Computer Networks.
3. Computer Networking: A Top-Down Approach.

### Online Resources

- [GeeksforGeeks CRC Tutorial](#)
- [IBM CRC Documentation](#)

### Class Notes

- Computer Network Class Notes